

# Android Automation Environment Setup and Appium Testing Report

---

## 1. Objective

The objective of this report is to demonstrate the complete setup of an Android mobile automation testing environment using **Java, Android Studio, and Appium**, and to validate automation testing on both **virtual (emulator)** and **real Android devices**.

This includes tool installation, environment configuration, device setup, APK installation using ADB, UI inspection, and automation of Android applications using **Java with UiAutomator2**.

---

## 2. System Environment

| Component       | Details                                  |
|-----------------|------------------------------------------|
| OS              | Windows                                  |
| Language        | Java                                     |
| IDE             | Eclipse (Maven Project)                  |
| SDK             | Android Studio & Platform Tools          |
| Automation Tool | Appium Server & Appium Inspector         |
| Devices         | Android Emulator and Real Android Device |

---

## 3. Software Installation & Configuration

### JDK

- Installed and verified using `java -version`
- Configured `JAVA_HOME` and system Path

### Eclipse IDE

- Installed Eclipse IDE for Java Developers
- Used for writing and running automation scripts

### Node.js

- Installed for running Appium server
- Verified using `node -v`

### Android Studio

- Installed with SDK and Emulator
- Configured `ANDROID_HOME`
- Added platform-tools and tools to system Path

### Appium Server GUI

- Installed and started at `127.0.0.1:4723`

### Appium Inspector

- Installed for identifying UI elements and resource IDs
-

## 4. Android Device Setup

### Android Emulator (AVD)

- Created using AVD Manager
- Verified using `adb devices`

### Real Android Device

- Enabled Developer Options and USB Debugging
  - Connected via USB and authorized
  - Verified using `adb devices`
- 

## 5. APK Installation Using ADB

### On Real Device

```
adb install <apk-file-path>
```

APK installed and launched successfully.

### On Emulator

Same ADB command used after emulator verification.

---

## 6. Appium Automation Configuration

### Common Desired Capabilities

- Platform: Android

- Automation: UiAutomator2
  - Device identified using UDID
  - Server: 127.0.0.1:4723
  - Command timeout: 60 seconds
- 

## 7. Automation – Android Calculator

### App Details Identified via Inspector

| Property | Value                              |
|----------|------------------------------------|
| Package  | com.coloros.calculator             |
| Activity | com.android.calculator2.Calculator |

### Test Steps

1. Open calculator
2. Perform 8 + 2
3. Click =
4. Read result
5. Validate result equals 10

**Result:** Test Passed successfully.

---

## 8. Automation – Phone Dialer

### App Details

| Property | Value                     |
|----------|---------------------------|
| Package  | com.google.android.dialer |
| Activity | GoogleDialtactsActivity   |

### Test Steps

1. Open dial pad
2. Enter number 09810012345
3. Click call button
4. Close session

**Result:** Dialer automation successful.

---

## 9. Verification Methods

- `java -version`
  - `node -v`
  - `adb devices`
  - Emulator launch
  - Appium server status
  - Successful script execution in Eclipse
-

## 10. Conclusion

This activity successfully demonstrates:

- Complete Android automation tool setup
- Proper environment variable configuration
- Testing on both emulator and real device
- APK installation using ADB
- UI element identification using Appium Inspector
- Automation of Android Calculator and Dialer using Java and UiAutomator2

The end-to-end workflow confirms the readiness of the system for Android mobile automation testing in a practical environment.